

PROGRAMMING REVIEW

Siripra Kingchan
06.03.2026

[

ENVIRONMENT SETUP

Scoop, GCC, and VS Code optimization.

DATA TYPES

Memory-aware typing and CP tricks.

VARIABLES

Lifecycle, scope, and naming conventions.

FUNCTIONS

Modular design and parameter efficiency.

]

THE CP TOOLCHAIN

CORE COMPONENTS



SCOOP

The command-line installer that manages paths and versions automatically.



GCC (MINGW-W64)

The compiler suite providing g++, gdb, and make for performance-critical execution.



VS CODE

The lightweight, extensible IDE for fast development and debugging.

WHY SETUP MATTERS

A robust local environment is your first competitive advantage.

1. CONSISTENCY

Ensuring your local compiler matches the online judge environment (GCC version, C++ standard).

2. SPEED & AUTOMATION

Quick compilation and automated testing against sample cases save precious minutes during contests.

SETUP_02

01

Installing Scoop
package manager

02

Using Scoop to
install VS Code
and C++

03

Installing essential
VS Code
extensions

04

Creating a C++
competitive
programming
template

05

Testing your setup

INSTALLING SCOOP

INSTALLATION STEPS (Window only)

Open PowerShell as non-administrator

1. Set Execution Policy:

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

2. Install Script:

```
irm get.scoop.sh | iex
```

3. Verify Installation:

```
scoop --version
```

WHY SCOOP?

- » **No Admin Rights:** Installs tools in your user directory, avoiding permission issues.
- » **Clean Environment:** Automatically manages system PATH variables for all installed tools.
- » **Portable Configuration:** Easy to replicate your setup across different machines.

PRO TIP:

Use `scoop update *` to keep all your competitive programming tools current with a single command.

SCOOP_03

VS CODE & GCC

INSTALLATION COMMANDS

1. Install VS Code:

```
scoop bucket add extras  
scoop install vscode
```

2. Install GCC Suite:

```
scoop install gcc
```

3. Verify Compiler:

```
g++ --version
```

MINGW-W64 COMPONENTS

- » **g++:** The primary C++ compiler used for building your solutions.
- » **gdb:** The GNU Debugger for step-by-step code execution and bug hunting.
- » **make:** A build automation tool for managing complex projects.

UPDATE CYCLE:

Keep your tools current to avoid bugs and gain performance improvements:
scoop update vscode
scoop update gcc

INTEG_04

ESSENTIAL EXTENSIONS

```
scoop install llvm # For clang compiler  
scoop install cmake # Build system  
scoop install ninja # Build system
```

All tools will be automatically added to your PATH.

1. C/C++ Extension Pack (Microsoft)

```
code --install-extension ms-vscode.cpptools-extension-pack
```

2. Code Runner

```
code --install-extension formulahendry.code-runner
```

3. Competitive Programming Helper (cph)

```
code --install-extension DivyanshuAgrawal.competitive-programming-helper
```

EXT 05

VS CODE CONFIGURATION

SETTINGS.JSON

```
{
  "code-runner.runInTerminal": true,
  "code-runner.saveFileBeforeRun": true,
  "editor.tabSize": 4,
  "editor.fontFamily": "Consolas, 'Courier New', monospace",
  "editor.fontSize": 14
}
```

WHY THESE SETTINGS?

1. TERMINAL EXECUTION

Setting `runInTerminal` to `true` allows you to provide interactive input (`cin`) directly in the VS Code terminal.

2. CONSISTENT FORMATTING

A `tabSize` of 4 ensures your code structure is clear and consistent, which is vital for debugging complex logic.

3. READABILITY

Monospaced fonts like `Consolas` ensure that every character occupies the same width, making it easier to spot alignment errors.

CONFIG_06

THE CP TEMPLATE

STANDARD BOILERPLATE

```
#include <bits/stdc++.h>
using namespace std;

void solve() {
    // Your solution here
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t = 1;
    // cin >> t;
    while(t--) {
        solve();
    }
}
```

KEY COMPONENTS

1. UNIVERSAL HEADER

`#include <bits/stdc++.h>` includes all standard libraries at once, saving time during contests.

2. FAST I/O

`sync_with_stdio(false)` and `cin.tie(NULL)` speed up input/output operations, essential for avoiding TLE.

3. MODULAR SOLVE()

Keeping logic in `solve()` makes it easier to handle multiple test cases and keeps `main()` clean.

PRO TIP:

Use `\n` instead of `endl` to avoid unnecessary buffer flushes, which further improves I/O speed.

TEMP_07

1

Create a new file:
problem.cpp

2

Copy your
template code

3

Solve a simple
problem:

4

```
void solve() {  
    int a, b;  
    cin >> a >> b;  
    cout << a+b << endl;  
}
```

5

Run with Code
Runner
(Ctrl+Alt+N)

6

Input test values
and verify output

Competitive Programming Workflow



Create a new file for
each individual problem



Begin with your
standardized code template



Analyze the problem
statement thoroughly



Implement your solution
within the solve() function



Validate your solution with
provided sample inputs



Submit your final,
tested solution

Additional Resources

- CP-Algorithms
- Codeforces
- AtCoder
- LeetCode
- HackerRank
- <https://toi-coding.informatics.buu.ac.th/>



DATA TYPES & MEMORY

MEMORY FUNDAMENTALS

BITS & BYTES

A **bit** is the smallest unit of data (0 or 1).

A **byte** consists of 8 bits and is the basic unit of addressable memory.

CP CONSTRAINTS

Standard memory limits in contests are usually **128MB** or **256MB**.

Exceeding this limit results in a **Memory Limit Exceeded (MLE)** error.

OPTIMIZATION TIP:

Select the smallest data type that safely holds your data to maximize memory efficiency.

MEMORY CAPACITY MAPPING

How many elements can you store in 128MB?

DATA TYPE	SIZE (BYTES)	MAX ELEMENTS (128MB)
int	4	~32,000,000
long long	8	~16,000,000
double	8	~16,000,000
char / bool	1	~128,000,000

Note: Always leave a safety buffer for other program overheads. Don't push the limit to exactly 100%.

MEM_08

INTEGER PRECISION

COMPARING TYPES

INT

Size: 4 Bytes (32-bit)

Range: $\pm 2,147,483,647$ ($\approx \pm 2e9$)

Use: Standard counters, indices, small values.

LONG LONG

Size: 8 Bytes (64-bit)

Range: $\pm 9,223,372,036,854,775,807$ ($\approx \pm 9e18$)

Use: Sums, products, large constraints.

THE LL SUFFIX RULE

Intermediate calculations use the type of the operands. Without **LL**, you may overflow before the assignment.

```
long long x = 1000000 * 1000000; // OVERFLOW!  
long long y = 1000000LL * 1000000; // OK
```

CP CRITICAL TIP

When calculating `a * b` where both are `int`, cast one to `long long` first:
`long long res = (long long)a * b;`

Note: Bitwise shifts also need suffixes: `1LL << 60`.

INT_09

FLOATING POINT

PRECISION COMPARISON

FLOAT (4 BYTES)

~7 decimal digits of precision. Often insufficient for complex geometry or scientific calculations.

DOUBLE (8 BYTES)

~15-16 decimal digits of precision. The **standard choice** for real numbers in competitive programming.

Note: For extreme precision, some environments support `long double` (12-16 bytes).

THE EQUALITY PITFALL

Never use `==` to compare floating point numbers due to rounding errors.

```
double a = 0.1 + 0.2;
if (a == 0.3) // This might be FALSE!
```

THE EPSILON SOLUTION

Compare the absolute difference against a very small value (EPS).

```
const double EPS = 1e-9;
if (abs(a - b) < EPS) {
    // Numbers are "equal"
}
```

FLOAT_10

BOOL & CHAR

BOOLEAN FOUNDATIONS

THE LOGICAL TYPE

Represents truth values: `true` (1) or `false` (0).

```
bool is_found = false;
```

THE 1-BYTE PARADOX

Technically, a boolean only needs 1 bit. However, CPUs address memory in **bytes**. To make access fast and efficient, C++ allocates a full **1 byte** (8 bits) for a single `bool`.

CHARACTER FOUNDATIONS

THE ASCII TYPE

Stores a single character using its integer ASCII value.

```
char grade = 'A'; // ASCII 65
```

MEMORY & RANGE

Occupies **1 byte**. Range is typically -128 to 127 (signed) or 0 to 255 (unsigned).

Tip: In C, `char` is often used as a very small integer type to save memory in massive arrays.

TYPE_11

STRING

CORE BASICS & STORAGE

- **Header:** `#include <string>`
- **O(1) Access:** `s[i]` or `s.at(i)`
- **O(1) Size:** `s.length()` or `s.size()`
- **Concatenate:** `s1 + s2` or `s1 += s2`

```
string s = "Hello";  
s += " World"; // "Hello World"  
char c = s[0]; // 'H'
```

SEARCH & MODIFICATION

- **Find:** `s.find("sub")` (returns `string::npos` if not found).
- **Substr:** `s.substr(pos, len)` ($O(len)$).
- **Sort:** Alphabetical character sorting.

```
if (s.find("abc") != string::npos) { ... }  
sort(s.begin(), s.end());  
s.erase(0, 5); // Remove first 5 chars
```

INPUT & PARSING

- **cin:** Reads until whitespace (space, tab, newline).
- **getline:** Reads until newline (full line).
- **stringstream:** For complex formatting/parsing.

```
getline(cin, s); // Read whole line  
stringstream ss(s);  
while (ss >> word) { ... }
```

Tip: Use `cin.ignore()` after `cin >> n` before `getline`.

CP CONVERSION & TRICKS

- **stoi / to_string:** String $\leftrightarrow$ Number conversion.
- **Frequency:** Character counting pattern.
- **Reverse:** `reverse(s.begin(), s.end());`

```
int n = stoi("123");  
freq[c - 'a']++; // Map 'a'-'z' to 0-25  
string r = to_string(456);
```

CP Tip: Always use `const string&` in function parameters for $O(1)$ speed.

ARRAY FOUNDATIONS

WHAT IS AN ARRAY?

An **array** is a collection of elements of the **same type** stored in **contiguous memory** locations.

```
// Declaration
int arr[5] = {10, 20, 30, 40, 50};
```

KEY PROPERTIES:

- **Fixed Size:** Defined at compile time (for static arrays).
- **0-Indexed:** The first element is at index 0.
- **Fast Access:** Accessing any element takes $O(1)$ time.

MEMORY ARCHITECTURE

Elements are laid out side-by-side in RAM. This allows the CPU to calculate the address of any element instantly.

0	1	2	3	4
10	20	30	40	50

Contiguous memory blocks in RAM

THE FOUNDATION

Arrays are the most fundamental data structure. They are the building blocks for strings, stacks, queues, and even more complex structures like hash tables.

ARRAY_21

ARRAY DECLARATION

SYNTAX & CONSTRAINTS

An array is a collection of elements of the **same type** stored in **contiguous memory**. The size must be a constant known at compile time.

```
// Declaration
int arr[10];
double scores[100];

// Initialization
int primes[] = {2, 3, 5, 7};
```

Static arrays cannot be resized once declared. For dynamic sizing, use `std::vector`.

0-INDEXING & OFFSETS

C++ uses **0-indexing**. The index represents the **offset** from the starting memory address of the array.

```
int arr[5] = {10, 20, 30, 40, 50};
cout << arr[0]; // 10 (Offset 0)
cout << arr[4]; // 50 (Offset 4)
```

UNDER THE HOOD

$\text{Address} = \text{Base} + (\text{Index} \times \text{Size})$

This simple math is why 0-indexing is used; it's the most efficient way for the CPU to calculate memory locations.

Danger: Accessing `arr[5]` is out-of-bounds and leads to undefined behavior!

MULTI-DIM ARRAYS

GRIDS & MATRICES

A 2D array is essentially an **array of arrays**. It is used to represent tabular data, grids, or matrices.

```
// Declaration: rows x cols
int grid[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Access: row 1, col 2
```

Memory is still contiguous; the compiler treats it as a single long block.

VISUAL REPRESENTATION

0,0 1	0,1 2	0,2 3
1,0 4	1,1 5	1,2 6
2,0 7	2,1 8	2,2 9

CP USE CASES

Game Boards: Chess, Tic-Tac-Toe, Maps.

DP Tables: Storing subproblem results.

Adjacency Matrices: Representing graphs.

Spatial Data: Coordinate systems.

GRID_23

ARRAY INITIALIZATION

STANDARD METHODS

Always initialize arrays to avoid "garbage values" from memory. Local arrays are NOT zeroed by default.

```
// Zero out all elements
int arr[100] = {0};

// Manual loop initialization
for (int i = 0; i < 100; i++) {
    arr[i] = -1;
}
```

Global arrays are automatically zeroed, but explicit initialization is safer.

THE MEMSET FUNCTION

For large arrays, `memset` (from `<cstring>`) is a high-performance way to set memory at the byte level.

```
#include <cstring>

int arr[1000];

memset(arr, 0, sizeof(arr));
memset(arr, -1, sizeof(arr));
```

MEMSET WARNING

`memset` works **byte-by-byte**. It is safe for `0` and `-1` with integers. Using it for other values (like `1`) will result in unexpected numbers!

Tip: For other values, use `std::fill(arr, arr + n, val);`

INIT_24

BOUNDS CHECK DANGER

OUT-OF-BOUNDS ACCESS

Accessing an index outside the range `[0, size-1]` is a critical error. C++ does **not** perform automatic bounds checking for performance reasons.

```
int arr[5] = {1, 2, 3, 4, 5};  
cout << arr[5]; // BUG: Index 5 is out-of-bounds!  
cout << arr[-1]; // BUG: Negative index!
```

CONSEQUENCES

- Runtime Error (RE):** Segmentation Fault (SIGSEGV).
- Undefined Behavior:** Silent data corruption or accessing other variables' memory.
- Wrong Answer (WA):** The program runs but produces incorrect results.

PREVENTION STRATEGIES

Always ensure your indices are within the valid range. Be especially careful with **off-by-one** errors in loops.

```
// Safe Loop Pattern  
for (int i = 0; i < size; i++) {  
    // Process arr[i]  
}
```

CP BEST PRACTICES:

- Padding:** Declare arrays slightly larger than needed (e.g., `arr[N+5]`).
- 1-Based Indexing:** If the problem uses 1-based indexing, declare `arr[N+1]` and use indices 1 to N.
- Vector Safety:** Use `v.at(i)` during debugging to catch out-of-bounds access immediately.

TYPE CAPACITY & MEMORY

MEMORY FUNDAMENTALS

BITS & BYTES

A **bit** is the smallest unit of data (0 or 1).

A **byte** consists of 8 bits and is the basic unit of addressable memory.

CP CONSTRAINTS

Standard memory limits in contests are usually **128MB** or **256MB**.

Exceeding this limit results in a **Memory Limit Exceeded (MLE)** error.

OPTIMIZATION TIP:

Select the smallest data type that safely holds your data to maximize memory efficiency.

MEMORY CAPACITY MAPPING

How many elements can you store in 128MB?

DATA TYPE	SIZE (BYTES)	MAX ELEMENTS (128MB)
int	4	~32,000,000
long long	8	~16,000,000
double	8	~16,000,000
char / bool	1	~128,000,000

Note: Always leave a safety buffer for other program overheads. Don't push the limit to exactly 100%.

MEM_08

TYPE CONVERSION

IMPLICIT (PROMOTION)

Automatic conversion by the compiler when a smaller type is assigned to a larger type.

```
int i = 42;
double d = i; // 42.0 (int → double)
```

Promotion Rule: In expressions with mixed types, the smaller type is promoted to the larger one.

```
int a = 5;
double b = 2.0;
auto res = a / b; // res is 2.5 (double)
```

EXPLICIT (CASTING)

Manual conversion using casting operators. Necessary when moving from larger to smaller types.

```
double pi = 3.14159;
int truncated = (int)pi; // 3
```

POTENTIAL DANGERS

Truncation: Losing decimal parts (double → int).

Precision Loss: Large long long to float .

Overflow: Large values into smaller containers.

CP Tip: Always cast explicitly when precision matters to avoid subtle bugs in geometric or mathematical problems.

CONV_13

CHAR ARITHMETIC

CHARACTERS AS NUMBERS

In C++, `char` is an integer type. Every character is mapped to a numeric value via the **ASCII** standard.

```
char c = 'A';  
int val = c; // val is 65  
  
char next = c + 1; // next is 'B' (66)
```

Common ASCII Values:

'0' - '9': 48 - 57

'A' - 'Z': 65 - 90

'a' - 'z': 97 - 122

ESSENTIAL CP TRICKS

1. DIGIT TO INTEGER

Convert a numeric character to its actual integer value by subtracting '0'.

```
char c = '7';  
int num = c - '0'; // num is 7
```

2. CASE TOGGING

Convert between upper and lower case using the constant difference (32).

```
char lower = 'a';  
char upper = lower - ('a' - 'A'); // 'A'
```

Note: While `toupper()` and `tolower()` exist, understanding the math is vital for custom transformations.

THE INDEXING TRICK

CONCEPTUAL MAPPING

Characters are stored as ASCII values. By subtracting the base character, we can map them to a 0-indexed range.

'a' - 'a'	→ 0
'b' - 'a'	→ 1
'c' - 'a'	→ 2
'z' - 'a'	→ 25

This works because ASCII values for 'a'-'z' are contiguous (97-122).

FREQUENCY ARRAY

Use a fixed-size array for O(1) character counting. This is much faster than using `std::map`.

```
string s = "olympiad";
int freq[26] = {0};

for(char c : s) {
    freq[c - 'a']++;
}
```

WHY USE THIS?

- Speed:** Array access is O(1), Map is O(log N).
- Memory:** Fixed 26 integers vs. dynamic nodes.
- Simplicity:** No extra headers or complex syntax.

TRICK_15

Type Best Practices

- *Choose the right type for your data*
- *Be aware of bounds and overflow*
- *Initialize variables before use*
- *Use descriptive variable names*
- *Consider memory constraints in competitive programming*

HANDS-ON EXERCISES

01

TYPE CONVERSION

Observe precision loss and overflow during numeric transitions.

02

ARRAY MANIPULATION

Create, populate, and analyze a 10-element data sequence.

03

STRING PROCESSING

Master character frequency mapping and string traversal.

EXERCISE 1:

Data Type Conversion

THE CHALLENGE

Task 1: Convert `int a = 5` to `float b` and back to `int c`. Observe any precision loss.

Task 2: Convert a large `long long big` to a smaller `int small`. What happens to the value during overflow?

```
int a = 5;
float b = a; // What is b?
int c = b; // What is c?

long long big = 2147483648LL;
int small = big; // What happens here?
```

EXERCISE 2:

ARRAY OPERATIONS

THE CHALLENGE

Task 1: Create an array of 10 integers initialized to 0.

Task 2: Fill the array with values 1-10.

Task 3: Calculate the sum and average of all elements.

Task 4: Find the maximum element in the array.

```
int arr[10] = {0};  
// Your code here to fill values 1-10  
// Your code here to calculate sum/average  
// Your code here to find maximum element
```

EXERCISE 3:

CHARACTER FREQUENCY

THE CHALLENGE

Task 1: Given the string `s = "competitive programming"` .

Task 2: Count the frequency of each letter using an array
`int freq[26] = {0}`

Task 3: Print letters that appear more than once.

```
string s = "competitive programming";  
int freq[26] = {0};  
  
// Your code here to count frequencies  
  
// Your code here to print duplicates
```

BREAK TIME

Recharge your mind & reset.

10 minute



Stay Hydrated



Stretch & Move



Review & Prepare Questions

VARIABLE BASICS

CORE COMPONENTS

TYPE

Defines what kind of data the variable can store (e.g., int, double).

NAME

The identifier used to access the stored data in your code.

VALUE

The actual data currently stored in the memory location.

ADDRESS

The physical location in the computer's memory (RAM).

ASSIGNMENT & NAMING

The `=` operator is used to store a value. The previous value is **overwritten**.

```
int score = 0; // Init
score = 10; // Overwrite
score = score + 5; // Update
```

DESCRIPTIVE NAMING

Avoid single letters except for loop indices. Clear names reduce bugs and improve readability.

Good: `playerScore`, `minDistance`
Bad: `ps`, `d1`

VAR_16

SAFE INITIALIZATION

THE "GARBAGE" DANGER

Local variables in C++ are **not** automatically zeroed. They contain whatever data was previously at that memory address.

```
void solve() {  
    int count; // Uninitialized!  
    count++; // UNDEFINED BEHAVIOR  
    cout << count; // Might print 4206912  
}
```

NON-DETERMINISTIC BUGS

Your code might work on your machine but fail on the judge because the "garbage" values are different. **Always initialize.**

BEST PRACTICES

1. Initialize at Declaration:

```
int total = 0;  
double average = 0.0;  
bool found = false;
```

2. Zero-Initialize Arrays:

```
int freq[26] = {0}; // All elements set to 0
```

CP Fact: Global and static variables are automatically initialized to zero by the compiler, but relying on this for local variables is a common source of errors.

INIT_17

NAMING RULES

STRICT C++ RULES

1. STARTING CHARACTERS

Must start with a letter (a-z, A-Z) or an underscore (_). Cannot start with a digit.

2. COMPOSITION

Can contain letters, digits, and underscores. No special characters like @, \$, or %.

3. CASE SENSITIVITY

`playerScore` and `playerscore` are two different variables.

4. RESERVED KEYWORDS

Cannot use C++ keywords like `int`, `while`, or `return` as names.

STYLE & CONSISTENCY

CAMELCASE

```
int currentScore = 0;
```

SNAKE_CASE

```
int current_score = 0;
```

BEST PRACTICES:

Use descriptive names (e.g., `min_dist` instead of `d`).

Avoid single letters except for loop indices (`i, j, k`).

Pick one style and stick to it throughout your project.

CONSTANTS

THE CONST KEYWORD

Constants are values that **cannot be changed** after initialization. They provide safety and clarify your intent to the compiler and other programmers.

```
const double PI = 3.14159;
const int MAX_N = 1e5 + 5;

// Error: cannot modify
PI = 3.14; // Compiler Error!
```

Why use them?

- Prevents accidental value changes.
- Makes code easier to maintain.
- Allows compiler optimizations.

CP STANDARDS

In competitive programming, we use constants for common values to avoid "magic numbers" in our code.

```
const int MOD = 1e9 + 7;
const int INF = 1e9;
const long long INF_LL = 1e18;
```

OPTIMIZATION BENEFITS

The compiler can replace constant identifiers with their actual values during compilation, leading to faster execution compared to using variables.

Tip: Use uppercase for constant names to distinguish them from regular variables.

CONST_19

VARIABLE SCOPE

LOCAL SCOPE

BLOCK VISIBILITY

Variables declared inside `{ }` are local to that block. They are created when the block is entered and destroyed when it is exited.

```
void solve() {  
    int x = 10; // Local to solve()  
    if (true) {  
        int y = 5; // Local to if block  
    }  
    // y is not accessible here
```

GLOBAL SCOPE

PROGRAM-WIDE ACCESS

Variables declared outside all functions are global. They are accessible from any part of the code.

```
int global_arr[1000005];  
  
void solve() {  
    global_arr[0] = 1; // OK  
}
```

CP CRITICAL: STACK OVERFLOW

The **Stack** (where local variables live) is small (~1-8MB). Large arrays declared locally will cause a crash. **Always declare large arrays globally** to place them in the static segment.

SCOPE_20

VARIABLE LIFETIME

AUTOMATIC LIFETIME

Standard local variables have **automatic** lifetime. They are created when the scope is entered and destroyed when it is exited.

```
void func() {  
    int x = 0; // Created  
    x++;  
    cout << x; // Prints 1  
} // x is DESTROYED here
```

Every time `func()` is called, a new `x` is created and initialized to 0.

STATIC LIFETIME

entire program duration

Variables declared with `static` persist for the entire program duration. They are initialized only once.

```
void func() {  
    static int count = 0;  
    count++;  
    cout << count;  
}
```

PERSISTENCE IN ACTION

1st call: Prints 1

2nd call: Prints 2

3rd call: Prints 3

CP Tip: Use static variables inside functions to maintain state without polluting the global namespace.

MEMORY MODEL

WHERE VARIABLES LIVE

1. THE STACK

Fast, automatic, but small (~1-8MB). Used for local variables and function calls. Exceeding this causes **Stack Overflow**.

2. THE HEAP

Large, manual (using `new`), but slower. Used for dynamic memory allocation.

3. STATIC SEGMENT

Where global and static variables live. Program-wide lifetime. Large capacity.

SEGMENT COMPARISON

SEGMENT	SPEED	SIZE	MANAGEMENT
Stack	Fastest	Small	Automatic
Heap	Slower	Large	Manual
Static	Fast	Large	Compiler

CP CRITICAL TIP:

Always declare large arrays (e.g., `int arr[1000000]`) as **global** or **static** to place them in the static segment and avoid stack overflow.

MODEL_23

BREAK TIME

Recharge your mind & reset.

10 minute



Stay Hydrated



Stretch & Move



Review & Prepare Questions

Function



FUNCTION ARCHITECTURE

CORE STRUCTURE

Functions are the building blocks of modular code. They encapsulate logic into reusable units.

```
int add(int a, int b) {  
    int res = a + b;  
    return res;  
}
```

- **Return Type:** The kind of data returned (int).
- **Name:** The identifier (add).
- **Parameters:** Inputs to the function (a, b).
- **Body:** The logic inside {}.

MODULAR DESIGN BENEFITS

1. ABSTRACTION

Hide complex logic behind a simple name. Call `isPrime(n)` without worrying about the implementation details.

2. REUSABILITY

Write once, use everywhere. Avoid duplicating code, which reduces the surface area for bugs.

3. TESTING & DEBUGGING

Small, focused functions are easier to test individually. If `gcd()` works, you can trust it in your main algorithm.

*SRP: A function should do **one thing** well.*

FUNC_24

DECL. VS DEF.

SIGNATURES VS. CODE

A function must be known to the compiler before it is called. This can be achieved through either a declaration or a definition.

```
// Declaration (Prototype)
int calculate(int x, int y);

// Definition (Implementation)
int calculate(int x, int y) {
    return x * y + 5;
}
```

The declaration only needs the signature (return type, name, and parameters).

WHY SEPARATE THEM?

FORWARD DECLARATION

Allows functions to call each other regardless of their order in the file.
Essential for **Mutual Recursion**.

CODE ORGANIZATION

In larger projects, declarations go into header files (`.h`) and definitions go into source files (`.cpp`).

Note: In single-file C++ solutions, definitions are usually placed before `main()` to avoid needing separate prototypes.

ORG_25

PARAMETER PASSING

PASS BY VALUE

The function receives a **copy** of the argument. Changes inside the function do not affect the original variable.

```
void inc(int x) { x++; }  
  
int a = 5;  
  
inc(a); // a is still 5
```

Safe for small types like `int` or `char`.

PASS BY REFERENCE (&)

The function receives **direct access** to the original variable. No copy is made, and changes affect the original.

```
void inc(int& x) { x++; }  
  
int a = 5;  
  
inc(a); // a is now 6
```

TLE WARNING!

Passing large objects like `vector<int>` or `string` by value triggers a **deep copy** every time the function is called. This can easily lead to **Time Limit Exceeded (TLE)** in competitive programming.

PARAM_26

CONST REFERENCE

BEST OF BOTH WORLDS

A **const reference** provides the speed of pass-by-reference without the risk of accidentally modifying the original data.

```
void print_vec(const vector<int>& v) {  
    for(int x : v) cout << x << " ";  
    // v[0] = 10; // Compiler Error!  
}
```

The compiler ensures the function only "reads" the data.

CP PERFORMANCE STANDARD

Always use **const reference** for passing large objects (vectors, strings, custom structs) to read-only functions.

PARAMETER COMPARISON

Pass by Value: Slow (copy), Safe (read/write copy).

Pass by Reference: Fast (no copy), Unsafe (read/write original).

Const Reference: Fast (no copy), Safe (read-only original).

Tip: For simple types (int, double), pass by value is usually faster than pass by reference due to pointer indirection overhead.

CONST_27

RETURN LOGIC

SENDING DATA BACK

The `return` statement exits the function and sends a value back to the caller.

```
int square(int x) {  
    return x * x;  
}
```

Type Matching: The value returned must match the function's return type. For functions that return nothing, use `void`.

```
void log(string msg) {  
    cout << msg << endl;  
    return; // Optional for void  
}
```

THE EARLY RETURN PATTERN

Exiting as soon as a condition is met simplifies logic and reduces nesting (e.g., base cases in recursion).

```
bool isEven(int n) {  
    if (n % 2 == 0) return true;  
    return false;  
}
```

GUARD CLAUSES

Check for invalid inputs or edge cases at the beginning of the function and return immediately. This makes the "happy path" easier to follow.

Note: In recursion, the base case is an early return that prevents infinite calls.

CLEAN DESIGN

FUNCTION OVERLOADING

Multiple functions can share the same name if they have **different parameter lists** (number or type of arguments).

```
int area(int side) { return side * side; }  
double area(double r) { return 3.14 * r * r; }  
int area(int l, int w) { return l * w; }
```

The compiler chooses the correct version based on the call signature.

SINGLE RESPONSIBILITY

The **Single Responsibility Principle (SRP)** states that a function should have one clear purpose.

WHY USE SRP?

Modularity: Easy to reuse in other parts of the solution.

Readability: Function names like `isPrime` or `gcd` explain the intent.

Maintenance: If there's a bug in the prime check, you only need to look at one small function.

Tip: If a function name has "And" in it (e.g., `inputAndProcess`), it might be doing too much!

DESIGN_29

SESSION SUMMARY

KEY TAKEAWAYS

DATA TYPES

Understand the range of `long long` and the precision pitfalls of `double`.

VARIABLES

Be mindful of scope and lifetime. Always initialize local variables to avoid garbage values.

FUNCTIONS

Use modular design and pass large objects by **const reference** to prevent TLE.

TASK: MODULAR PRIME CHECKER

1. Use the standard CP template.
2. Create a function `bool isPrime(long long n)`.
3. Implement the $O(\sqrt{N})$ primality test.
4. In `solve()`, read `T` test cases and output "YES" or "NO".

```
bool isPrime(long long n) {
    if (n < 2) return false;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

DONE_30

EXERCISE 1:

VARIABLE SCOPE MASTERY

PRACTICAL TASKS

Task 1: Inside `scopeDemo()` , attempt to access `mainLocal` from `main()` . What error occurs?

Task 2: Modify the global `global` variable inside `scopeDemo()` .

Advanced: Declare a local `global` variable inside the function. Which one wins?

```
int global = 10;

void scopeDemo() {
    // 1. Try to access 'mainLocal'
    // 2. Modify 'global'
}

int main() {
    int mainLocal = 20;
    scopeDemo();
    return 0;
}
```

EXERCISE 2:

VARIABLE LIFETIME & STATIC

PRACTICAL TASKS

Task 1: Use the `static` keyword to create a counter that persists across function calls.

Task 2: Call `counterFunction()` 5 times in `main()` and observe the output.

Advanced: Compare behavior with a regular `int count = 0` local variable.

```
void counterFunction() {  
    // Implement persistent counter here  
    // Print: "Call count: X"  
}  
  
int main() {  
    for(int i=0; i<5; i++) {  
        counterFunction();  
    }  
    return 0;  
}
```

EXERCISE 3:

SHADOWING & SCOPE OPERATOR

PRACTICAL TASKS

Task 1: Create a nested block `{ ... }` with another `value` .

Task 2: Access the global `value` using `::value` .

Challenge: Print all 3 levels (Global, Local, Shadow) from the innermost scope.

```
int value = 100; // Global
int main() {
    int value = 50; // Local
    {
        int value = 10; // Shadow
        // Access all 3 levels here
    }
    return 0;
}
```

FLOW CONTROL



FLOW CONTROL

THE LOGIC PILLARS

1. SEQUENTIAL

Instructions are executed in the exact order they appear, from top to bottom.

2. CONDITIONAL

The program branches based on the result of a logical test (if-else).

3. ITERATIVE

A block of code is repeated multiple times based on a condition (loops).

ALGORITHMIC FOUNDATION

These three structures form the "skeleton" of every algorithm. Complexity arises from combining them.

THE PROGRAMMER'S GOAL

To manipulate the execution flow of a program to solve a specific problem with the highest efficiency.

Note: This session focuses on the "Conditional" and "Iterative" pillars.

FLOW_02

IF-ELSE LOGIC

BRANCHING FOUNDATIONS

The `if` statement evaluates a condition. If true, the code block is executed. Otherwise, the `else` block is executed.

```
if (score >= 50) {  
    cout << "Pass";  
} else {  
    cout << "Fail";  
}
```

WHAT IS "TRUE"?

In C++, **any non-zero value** is true. **Zero** is false. This is why `if(5)` is true and `if(0)` is false.

BOOLEAN EXPRESSIONS

Use comparison operators to create conditions that evaluate to true or false.

```
a == b; // Equality  
a != b; // Inequality  
a > b; // Greater than  
a < b; // Less than  
a >= b; // Greater than or equal  
a <= b; // Less than or equal
```

Note: The assignment operator `=` is often confused with the equality operator `==`. This is a common bug.

IF_03

ELSE-IF CHAINS

MULTI-WAY BRANCHING

When you have multiple mutually exclusive conditions, use an `else if` chain. The compiler checks each condition in order.

```
if (score >= 80) {  
    grade = 'A';  
} else if (score >= 70) {  
    grade = 'B';  
} else if (score >= 60) {  
    grade = 'C';  
} else {  
    grade = 'F';  
}
```

Only the first block with a true condition is executed.

EFFICIENCY & CLARITY

MUTUALLY EXCLUSIVE LOGIC

Once a condition is met, the rest of the chain is skipped. This is more efficient than multiple independent `if` statements and more readable than nested `if-else` blocks.

BEST PRACTICES:

Order conditions from most specific to most general.

Always include a final `else` block to handle unexpected cases.

Keep chains concise to maintain readability.

CHAIN_04

COMPLEX DECISIONS

LOGICAL OPERATORS

Combine multiple conditions into a single boolean expression using logical operators.

OPERATOR	MEANING	EXAMPLE
&&	AND	(a > 0 && a < 10)
	OR	(a == 0 a == 1)
!	NOT	!(a == 5)

```
if (age >= 18 && has_id) {  
    cout << "Entry allowed";  
}
```

NESTED CONDITIONALS

Hierarchical decisions can be made by placing `if` statements inside other `if` statements.

```
if (is_weekend) {  
    if (is_sunny) {  
        cout << "Go to beach";  
    } else {  
        cout << "Stay home";  
    }  
}
```

AVOID "ARROW CODE"

Deep nesting makes code hard to read. Use **Logical Operators** or **Guard Clauses** (early returns) to flatten your logic and improve clarity.

NEST_05

SWITCH STATEMENT

DISCRETE BRANCHING

The `switch` statement evaluates an expression and matches it against multiple `case` labels. It's cleaner than long `if-else` chains for discrete values.

```
switch (grade) {  
    case 'A':  
        cout << "Excellent";  
        break;  
    case 'B':  
        cout << "Good";  
        break;  
    default:  
        cout << "Keep trying";  
}
```

The `break` statement is vital to prevent "fall-through" to the next case.

SWITCH VS. IF-ELSE

WHEN TO USE SWITCH?

Discrete Values:

When comparing a single variable against multiple constants (int, char, enum).

Readability: Cleaner syntax for many branches.

Performance:

Compilers can often optimize switch statements into jump tables, making them faster than long if-else chains.

Note: Switch cannot be used for ranges (e.g., `score > 80`) or floating-point values.

SWITCH_06

IF-ELSE

```
void move(int& x, int& y, char dir) {  
    if (dir == 'N') {  
        y++;  
    } else {  
        if (dir == 'S') {  
            y--;  
        } else {  
            if (dir == 'E') {  
                x++;  
            } else {  
                if (dir == 'W') {  
                    x--;  
                }  
            }  
        }  
    }  
}
```

SWITCH

```
void move(int& x, int& y, char dir) {  
    switch (dir) {  
        case 'N': y++; break;  
        case 'S': y--; break;  
        case 'E': x++; break;  
        case 'W': x--; break;  
    }  
}
```

TERNARY OPERATOR

CONCISE LOGIC

The **ternary operator** (`? :`) is a shorthand for a simple if-else statement that returns a value.

```
// Syntax
condition ? expr_if_true : expr_if_false;
```

IF-ELSE VS. TERNARY

If-Else:

```
int max_val;
if (a > b) max_val = a;
else max_val = b;
```

Ternary:

```
int max_val = (a > b) ? a : b;
```

CP USE CASES

Use the ternary operator for simple assignments to keep your code clean and readable.

```
// Finding min/max
int min_val = (a < b) ? a : b;

// Conditional output
cout << (is_even ? "Even" : "Odd") << endl;
```

Nested Ternary: You can nest them, but be careful with readability.

```
string res = (n > 0) ? "Pos" : (n < 0) ? "Neg" : "Zero";
```

Tip: Always use parentheses around the condition to improve clarity.

TERN_07

CONTROL PITFALLS

ASSIGNMENT VS. EQUALITY

Using `=` instead of `==` inside an `if` statement is a classic bug. It assigns the value and evaluates the result.

```
int x = 0;
if (x = 5) { // BUG: Assigns 5 to x
    // This block ALWAYS runs
    // because 5 is non-zero (true)
}
```

YODA CONDITIONS

Some programmers use `if (5 == x)`. If you accidentally type `if (5 = x)`, the compiler will throw an error because you can't assign to a literal.

THE MISSING BRACES TRAP

Without braces, only the **next single statement** is part of the `if` block. This leads to subtle logic errors.

```
if (score > 100)
    cout << "High Score!";
    bonus = 10; // BUG: Always runs!
```

The Dangling Else: An `else` always attaches to the **nearest** preceding `if` unless braces are used.

```
if (a > 0)
    if (b > 0) cout << "Both positive";
    else cout << "a is negative?"; // NO!
```

*Best Practice: **Always use braces**, even for single-line statements, to ensure clarity and prevent bugs.*

BUG_08

LOGIC OPTIMIZATION

SHORT-CIRCUITING

C++ evaluates logical expressions from left to right and stops as soon as the result is certain.

```
// If 'ptr' is null, 'ptr->val' is NOT evaluated
if (ptr != nullptr && ptr->val > 0) { ... }

// If 'a' is true, 'expensive_func()' is NOT called
if (a || expensive_func()) { ... }
```

SAFETY & SPEED

1. **Safety:** Prevent out-of-bounds or null-pointer errors by checking the condition first.
 2. **Speed:** Place the "cheapest" or "most likely to fail" condition first in `&` chains.
- `&`

BRANCH PREDICTION

Modern CPUs try to guess which way a branch will go. A correct guess is fast; a wrong guess (misprediction) is expensive.

```
// Sorted data is faster to process
// because the branch is highly predictable.
for (int x : sorted_data) {
    if (x < threshold) count++;
}
```

CP OPTIMIZATION TIPS:

- Use `__builtin_expect` for critical hot paths (rare).
- Prefer bitwise operations for simple flags to avoid branches.
- Keep the "happy path" straight to help the predictor.

OPT_09

BREAK TIME

Recharge your mind & reset.

10 minute



Stay Hydrated



Stretch & Move



Review & Prepare Questions

CONTROL SUMMARY

DECISION TOOLS

IF-ELSE CHAINS

The primary tool for general branching and range-based conditions.

SWITCH STATEMENT

Optimized for discrete values (int, char). Use jump tables for speed.

TERNARY OPERATOR

Concise syntax for simple conditional assignments and outputs.

BEST PRACTICES

CP LOGIC CHECKLIST

Always use braces `{ }` to prevent logic bugs.

Avoid deep nesting; use guard clauses to flatten code.

Use logical operators (`&&` , `||`) to combine tests.

Be mindful of `=` vs `==` in conditions.

Prefer switch for many discrete constant cases.

NEXT SECTION: **LOOPS & ITERATION**

DONE_10

POWER OF REPETITION

WHY LOOPS?

1. AUTOMATION

Execute the same block of code multiple times without manual duplication.

2. SCALABILITY

Handle 10 items or 10,000,000 items with the same few lines of code.

3. EFFICIENCY

Reduce code size, minimize errors, and improve maintainability.

MANUAL VS. ITERATIVE

Imagine printing "Hello" 100 times. Manual duplication is impossible to maintain.

```
// Manual (Bad)
cout << "Hello";
cout << "Hello";
... // many more
```

```
// Iterative (Good)
for (int i = 0; i < 100;
i++) {
    cout << "Hello";
}
```

THE CORE OF CP

Iteration is the heart of data processing. Almost every competitive programming problem requires some form of looping to process input or explore a state space.

THE WHILE LOOP

CONDITION-FIRST

The `while` loop repeats a block of code as long as a specified condition remains **true**. The condition is checked **before** each iteration.

```
int i = 0;
while (i < 5) {
    cout << i << " ";
    i++;
}
```

If the condition is false initially, the loop body never executes.

CP APPLICATIONS

Ideal for situations where the number of iterations is not known in advance, such as reading input until the end of a file.

```
int n;
while (cin >> n && n != 0) {
    // Process n until 0 is entered
}
```

INFINITE LOOP RISK

Ensure that the loop condition eventually becomes **false**. Forgetting to update the control variable (like `i++`) will cause the program to hang.

LOOP_12

DO-WHILE LOOP

EXECUTE THEN CHECK

The do-while loop is a post-test loop. It executes the code block **first**, then checks the condition.

```
do {  
    // Code to execute  
} while (condition);
```

GUARANTEED EXECUTION

execute its block

Unlike the while loop, the do-while loop is guaranteed to **at least once**, even if the condition is false from the start.

PRACTICAL USE CASE

Ideal for scenarios where you need to perform an action before validating it, such as input validation.

```
int n;  
do {  
    cout << "Enter positive number: ";  
    cin >> n;  
} while (n <= 0);
```

KEY DIFFERENCES:

While: Check → Execute (0 or more times).

Do-While: Execute → Check (1 or more times).

Syntax: Don't forget the semicolon ; after the while condition!

LOOP_13

THE FOR LOOP

LOOP ANATOMY

The `for` loop is the most structured way to repeat code. It combines initialization, condition, and update in one line.

```
for (int i = 0; i < 5; i++) {  
    cout << i << " ";  
}
```

THE THREE PARTS

Initialization: Runs once at the start (`int i = 0`).

Condition: Checked before each body run (`i < 5`).

Update: Runs after each body run (`i++`).

COMMON PATTERNS

For loops are highly flexible. You can count up, count down, or skip values.

```
// Counting Down  
for (int i = 10; i > 0; i--) { ... }  
  
// Skipping Values  
for (int i = 0; i < 100; i += 2) { ... }
```

CP Tip: Use `for` loops for fixed-range iterations and traversing arrays or vectors where the size is known.

```
for (int i = 0; i < n; i++) {  
    cin >> arr[i];  
}
```

Note: The control variable `i` is local to the loop scope.

RANGE-BASED FOR

MODERN ITERATION

Introduced in C++11, the **range-based for loop** provides a cleaner way to iterate over all elements in a container (arrays, vectors, etc.).

```
vector<int> v = {1, 2, 3, 4, 5};  
for (int x : v) {  
    cout << x << " ";  
}
```

WHY USE IT?

Readability: Much cleaner than index-based loops.

Safety: No risk of "off-by-one" or out-of-bounds errors.

Flexibility: Works with any container that supports iterators.

ADVANCED USAGE

Use **references** to avoid copying large objects and **const** to prevent accidental modifications.

```
// Use & to modify elements  
for (auto &x : v) x *= 2;  
  
// Use const & for efficiency  
for (const auto &x : v) {  
    cout << x << endl;  
}
```

WHEN TO USE INDEX-BASED?

When you need the **index** (i) for calculations.

When you need to iterate in **reverse**.

When you need to skip elements or change the step size.

NESTED LOOPS

2D GRID TRAVERSAL

Nested loops are used to iterate over multi-dimensional data structures like grids, matrices, or coordinate systems.

```
for (int i = 0; i < rows; i++) {  
    for (int j = 0; j < cols; j++) {  
        cout << grid[i][j] << " ";  
    }  
    cout << endl;  
}
```

The inner loop completes all its iterations for every single iteration of the outer loop.

TIME COMPLEXITY

THE $O(N \times M)$ RULE

If the outer loop runs **N** times and the inner loop runs **M** times, the total number of operations is **$N \times M$** . For square grids, this is **$O(N^2)$** .

CP WARNING:

If $N = 10^5$, an $O(N^2)$ loop will perform 10^{10} operations. This will exceed the typical 1-second time limit (TLE). Always calculate the total operations before nesting!

NEST_16

MID-LOOP CONTROL

THE BREAK STATEMENT

The `break` statement immediately terminates the loop and moves execution to the statement following the loop.

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) break;  
    cout << i << " ";  
}  
// Output: 1 2 3 4
```

EARLY EXIT

Use `break` when you've found what you're looking for (e.g., searching an array) to save time and avoid unnecessary iterations.

THE CONTINUE STATEMENT

The `continue` statement skips the rest of the current iteration and jumps to the next one (the update step in a for loop).

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) continue;  
    cout << i << " ";  
}  
// Output: 1 2 4 5
```

CP FILTERING PATTERN:

Skip invalid inputs or edge cases.

Reduce nesting by using `continue`

instead of wrapping the whole body in an

Useful in grid traversals to skip out-of-bounds cells.

INFINITE LOOPS

ACCIDENTAL CAUSES

An infinite loop occurs when the loop condition **never** becomes false. This is usually a bug.

```
// Bug 1: Missing Update
int i = 0;
while (i < 5) {
    cout << i; // i never changes!
}

// Bug 2: Impossible Condition
for (int i = 1; i > 0; i++) {
    // i will always be > 0
}
```

DETECTION

In CP, this leads to **Time Limit Exceeded (TLE)**. Locally, your program will hang and consume 100% of a CPU core.

INTENTIONAL USE

Sometimes, an infinite loop is intentional, especially when the exit condition is complex or checked mid-body.

```
while (true) {
    int n;
    if (!(cin >> n)) break;
    if (n == -1) break;
    // Process n...
}
```

MANAGING "WHILE(TRUE)":

Always have a clear `break` or `return` path.
Useful for interactive programs or complex input patterns.

Can be cleaner than a complicated condition in the `while` header.

INF_18

LOOP EFFICIENCY

COMMON INEFFICIENCIES

Avoid redundant work inside the loop body. Every extra operation is multiplied by the number of iterations.

```
// Inefficient: strlen() called every time
for (int i = 0; i < strlen(s); i++) { ... }

// Efficient: Call once outside
int n = strlen(s);
for (int i = 0; i < n; i++) { ... }
```

THE INNER LOOP RULE

In nested loops, focus 90% of your optimization effort on the **innermost loop**. This is where the program spends the vast majority of its time.

PERFORMANCE TUNING

Pre-calculating values and simplifying logic can significantly reduce the constant factor of your algorithm.

```
// Pre-calculate squares to avoid multiplication
int sq[1001];

for (int i = 0; i <= 1000; i++) sq[i] = i * i;

// Use pre-calculated values in hot loop
for (int i = 0; i < n; i++) {
    if (val == sq[i]) { ... }
}
```

OPTIMIZATION CHECKLIST:

Move invariant calculations outside the loop.

Avoid heavy I/O (like `endl`) inside loops; use `"\n"`.

Use `reserve()` for vectors to avoid reallocations.

Prefer prefix `++i` over postfix `i++` for iterators.

PERF_19

Practice Time

ARRAY SUM PAIRS

Write a function to count all pairs in an array that sum to a given target.

```
Input: [1, 5, 3, 3, 4, 2], target = 6  
Pairs: [(1,5), (3,3), (4,2)]  
Count: 3
```

THINK ABOUT:

- **Data types:** Correct types for large targets.
- **Function parameters:** Passing by reference for efficiency.
- **Algorithm complexity:** Can you do it better than $O(N^2)$?
- **Edge cases:** Empty arrays, duplicates, or no pairs found.

STRING TRANSFORMATION

Write a function that transforms a string by:

1. Converting to uppercase
2. Removing all spaces
3. Reversing the result

```
Input: "Hello World"
```

```
Output: "DLROWOLLEH"
```

THINK ABOUT:

- **Pass by reference vs value:** Speed considerations.
- **String operations:** Using `std::reverse` and `std::toupper`.
- **Function composition:** How to structure the steps cleanly.
- **Efficiency:** Minimizing string copies.

PRIME COUNTER

Write a function that counts how many prime numbers exist below n.

```
Input: 10
```

```
Output: 4 (2, 3, 5, 7)
```

THINK ABOUT:

- **Helper functions:** Using `isPrime(x)` .
- **Algorithm efficiency:** $O(N\sqrt{N})$ vs $O(N \log \log N)$ Sieve.
- **Variable scope:** Managing loop variables.
- **Memory usage:** Storing prime flags.

ARRAY ROTATION

Write a function to rotate an array to the right by k steps.

```
Input: [1, 2, 3, 4, 5, 6, 7], k = 3
```

```
Output: [5, 6, 7, 1, 2, 3, 4]
```

THINK ABOUT:

- **In-place vs new array:** Memory trade-offs.
- **Modular arithmetic:** Handling $k > \text{array size}$
- **Edge cases:** $k=0$, $k=\text{size}$, or empty arrays.
- **Optimization:** Can you do it in $O(N)$ time and $O(1)$ space?

Challenge Problem

MATRIX SPIRAL TRAVERSAL

Write a function that returns all elements of an $m \times n$ matrix in spiral

order.

```
[ 1, 2, 3 ]  
[ 4, 5, 6 ]  
[ 7, 8, 9 ]
```

Output: [1, 2, 3, 6, 9, 8, 7, 4, 5]

THINK ABOUT:

- **Direction tracking:** Right → Down → Left → Up.
- **Boundary handling:** Updating top, bottom, left, right limits.
- **Loop termination:** When all elements are visited.
- **Edge cases:** Empty matrix or single row/column.

CONTINUING YOUR LEARNING

PRACTICE PLATFORMS

-  **Codeforces:** Best for algorithmic contests.
-  **LeetCode:** Data structures & interview prep.
-  **AtCoder:** High-quality math problems.
-  **HackerRank:** Domain-specific challenges.

ESSENTIAL READING

-  **Competitive Programming 4**
by Steven Halim
-  **CP Handbook**
by Antti Laaksonen
-  **USACO Guide:** Comprehensive online curriculum.

COMMUNITY & GROWTH

-  Join local programming contests.
-  Participate in online forums (Codeforces, Reddit).
-  Upsolve problems after every contest.

LOOPING SUMMARY

ITERATION TOOLS

WHILE / DO-WHILE

Best for unknown iteration counts or input-driven logic.

FOR LOOP

The standard for fixed-range iteration and index-based access.

RANGE-BASED FOR

Modern, safe, and readable traversal of STL containers.

BREAK / CONTINUE

Fine-grained control to terminate or skip iterations mid-loop.

ALGORITHMIC IMPACT

COMPLEXITY & TLE

Single Loop: $O(N)$ complexity.

Nested Loops: $O(N^2)$ or $O(N \times M)$.

CP Limit: $\sim 10^8$ operations per second.

$N = 10^5 \rightarrow O(N^2)$ will likely TLE.

DONE_20

SUMMARY

KEY TAKEAWAYS

CONTROL FLOW

Master branching with `if-else` and `switch`. Use short-circuiting for safety and speed.

ITERATION

Understand the anatomy of `for` and `while` loops. Be mindful of $O(N^2)$ complexity in nested loops.

STORGE

Use arrays for fixed-size data and
Always check your bounds!

FINAL CHALLENGE

TASK: 2D GRID TRAVERSAL

1. Read an $N \times M$ grid of integers.
2. Calculate the sum of all elements.
3. Find the maximum value in the grid.
4. Output the sum, max value, and its (row, col) coordinates.

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < m; j++) {  
        cin >> grid[i][j];  
        sum += grid[i][j];  
        if (grid[i][j] > max_val) { ... }  
    }  
}
```

Congratulations! You've completed the Day 1 Programming Review.

DONE

THANK YOU!

Questions?

Happy coding ^ ^